Advanced Computer-Aided VLSI System Design
Midterm Project: Vision Transformer Patch Embedding Accelerator

TA: 邱嚴盛 r14943004@ntu.edu.tw **Due Tuesday, May 12th, 13:59**

TA: 蔡岳峰 f12943014@ntu.edu.tw

Data Preparation

Unpack 1142_midterm.tar with the following command

```
tar -xvf 1142_midterm.tar
```

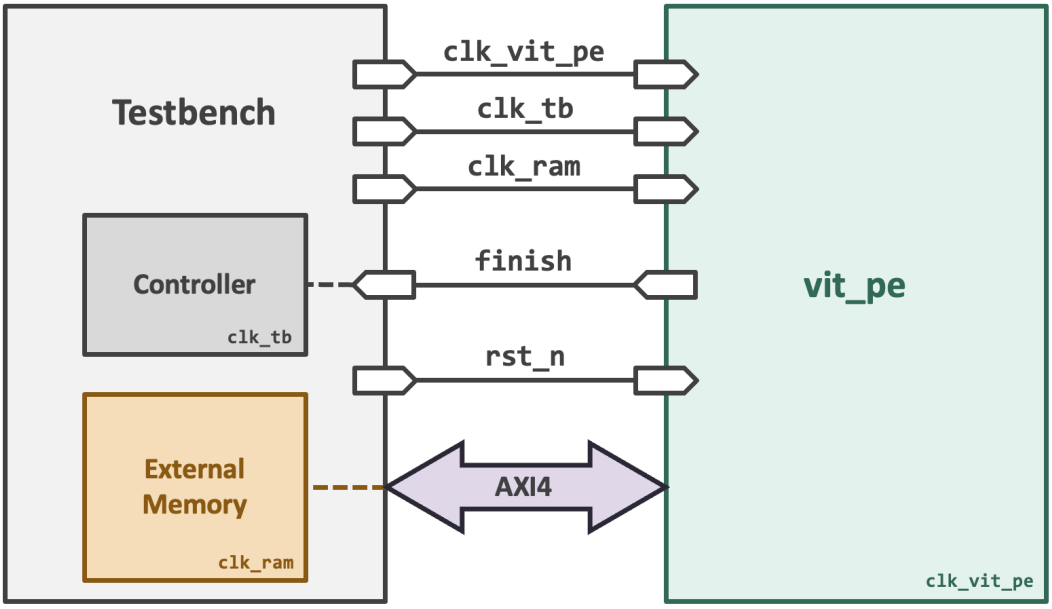
Directory	File	Description
00_TESTBED	vit_pe_tb.v	Testbench
00_TESTBED /axi_ram	axi_ram.v	External memory
00_TESTBED /patterns /Pi (i=0, 1, 2, 3)	Pi_<class>.png	Image for visual reference
	Pi_picture.dat	Input image in hex
	Pi_weights.dat	Parameters in hex
	Pi_golden.dat	Expected output results
01_RTL	vit_pe.v	Your design
	rtl.f	File list for RTL simulation
	01_run.tcsh	Command for RTL simulation
02_SYN	rtl_to_syn.v	File list for synthesis
	vit_pe_syn.sdc	Constraints for synthesis
	vit_pe_syn.tcl	TCL script for synthesis
03_GATE	netlist.f	File list for gate-level simulation
	03_run.tcsh	Command for gate-level simulation
	03_fsdb2vcd.tcsh	Command for fsdb2vcd conversion

Introduction

In this midterm project, you are asked to design a **Vision Transformer Patch Embedding Accelerator**. The input image, projection weights, quantization scale factors, CLS token, and position embeddings are stored in the external memory. There are three time domains, which are for control signal, for memory, and for your design, respectively. Your design needs to fetch the data and parameters via the AXI4 protocol, perform linear projection and INT8 quantization, and fuse the sequence with the CLS token and position embeddings. The final combined token sequence must

then be written back to the memory via the AXI protocol. After storing all results back to the memory, raise the finish signal for three cycles. The testbench will then verify the correctness of your output and terminate the simulation.

Block Diagram



Specifications

1. Top module name: **`vit_pe`**
2. Input/output description:

Signal Name	I/O	Width	Simple Description
`clk_tb`	I	1	• Primary clock for the testbench **(positive edge-triggered)** • Inputs are synchronized with a half-cycle delay
`clk_vit_pe`	I	1	• Design clock for the accelerator **(positive edge-triggered)**
`clk_ram`	I	1	• Clock for AXI channels and memory access **(positive edge-triggered)** • All AXI inputs/outputs must be synchronized with this clock's rising edge
`rst_n`	I	1	• **Active-low synchronous reset**
`finish`	O	1	• Computation completion flag • Must remain HIGH for at least three `clk_tb` cycles to signal termination
`awaddr`	O	15	• Address Channel (AW/AR)

araddr			• Specifies the start address of the first transfer in a burst
awlen **arlen**	O	8	• Burst Length • Number of data transfers within a burst ($N = len + 1$)
awsize **arsize**	O	3	• Burst Size • Specifies the size (bytes) of each individual transfer
awburst **arburst**	O	2	• Burst Type • Determines how the address increments for each transfer
awvalid **arvalid**	O	1	• Address Valid • Indicates that the address and control information are valid
awready **arready**	I	1	• Address Ready • Indicates that the slave is ready to accept the address
wdata **rdata**	O I	8	• Data Channel (W/R) • Write/Read data payload
wlast **rlast**	O I	1	• Last Transfer Indicator • Indicates the last transfer in a burst
wvalid **rvalid**	O I	1	• Data Valid • Indicates that the data/strobe is valid
wready **rready**	I O	1	• Data Ready • Indicates that the master/slave can accept data
wstrb	O	1	• Write Strobes • Indicates which byte lanes contain valid data
bresp **rresp**	I	2	• Response Status • Status of the write/read transfer (e.g., `2'b00` for OKAY)
bvalid **bready**	I O	1	• Write Response Channel (B) • Handshake signals for the write response

3. `rst_n` is synchronized with the positive edge of `clk_vit_pe`.
4. Test patterns (Image and Weights) are pre-loaded into the external memory after the reset signal is de-asserted.
5. All AXI input and output signals must be strictly synchronized with the positive edge of `clk_ram`.
6. **CDC techniques should be properly implemented.**
7. No explicit start signal is provided in this project; the accelerator begins operations autonomously once the memory is ready post-reset.
8. The `finish` signal must be synchronized with `clk_tb` and remain HIGH for three cycles to indicate the end of the entire computation task.
9. DesignWare and ADFP SRAM are allowed.

Design Description

1. Image Fetch & Linear Projection

The input image data should be divided into 64 4×4 patches, flattened into 1D vectors, and stacked into a 64×48 matrix where each element is an Unsigned 8-bit integer. The projection weights are represented as a 48×64 matrix consisting of Signed 8-bit integers. The engine must perform matrix multiplication to project the input patches into a 64× 64 embedding matrix.

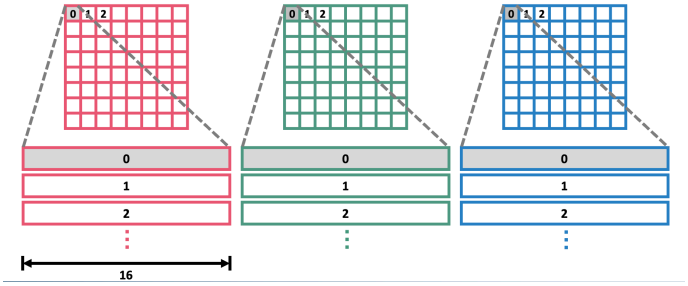
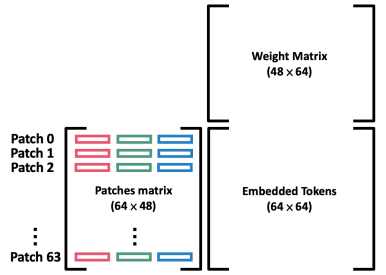


Image patches flattening



Linear Projection

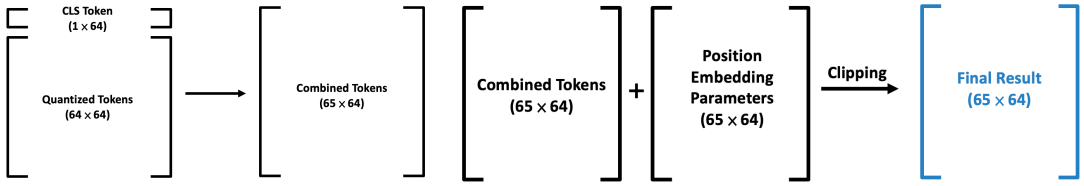
2. Quantization

After linear projection, the embedded tokens must be quantized back to 8-bit integers. We apply a per-dimension (per-channel) quantization scheme. For each of the 64 embedding dimensions (a column in the 64×64 matrix), there is a corresponding Unsigned 32-bit fixed-point scale factor. You must multiply the partial sum by its dedicated scale factor, and apply round-to-nearest logic. To preserve precision for subsequent operations, intermediate clipping is deferred at this stage.

$$j\text{-th_column}_{\text{INT8}} = \text{round}\left(j\text{-th_column} \cdot \text{scale_factor}[j]\right), \ j = 0, 1, 2, \cdots, 63$$

3. Sequence Construction & Position Embedding

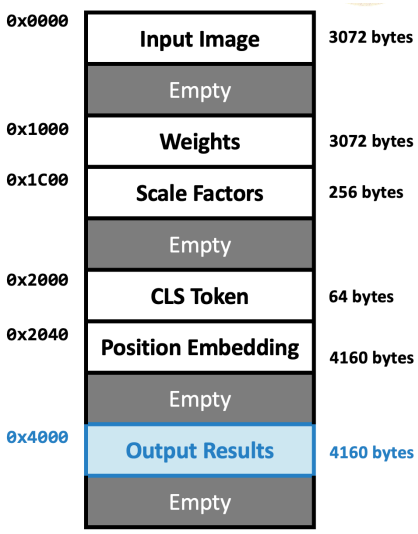
To retain spatial awareness and class information (since Transformers lack inherent sequence order), two components are added to the quantized sequence. First, a learnable CLS Token (a 1×64 row vector) is prepended to the 64×64 matrix as Token 0, creating a combined token sequence of size 65×64. Second, a 65×64 Position Embedding matrix is added element-wise to this combined sequence. After adding the position embeddings to the sequence, a final saturation step is required. The summed results must be strictly clipped into the Signed INT8 range (making positive values beyond 127 be 127, and negative values beyond -128 be -128).



CLS Token prepending Position Embedding

5. Memory Mapping

The external memory has a word size of 8 bits and can store up to 32768 words (32KB). To simplify notation, the *i*-th word in the memory is denoted as MEM[*i*]. All data are stored in raster-scan ordering.



Memory Map

5.1. Input Image

The input image data are stored from MEM[0x0000] to MEM[0x0BFF] (3072 bytes).

5.2. Model Parameters

Model parameters are strictly mapped to their designated base addresses:

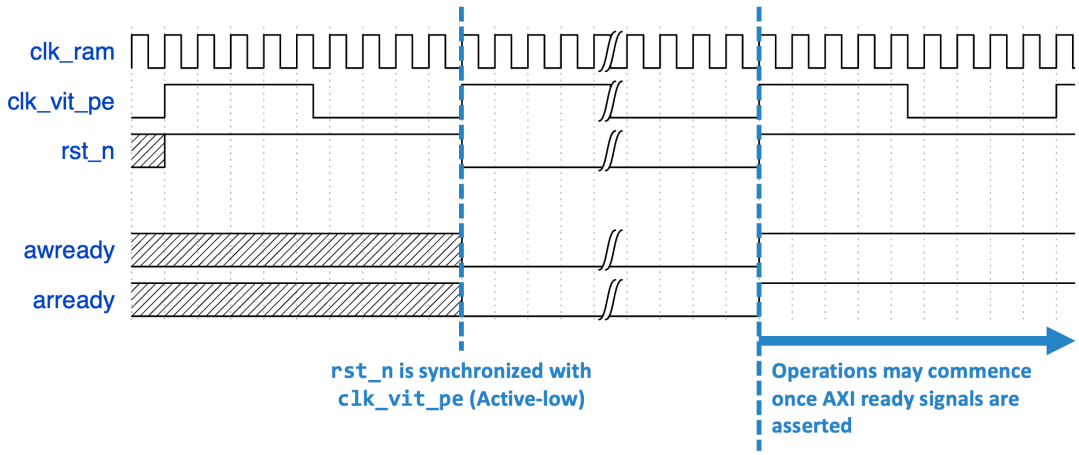
- **Weights:** Stored from MEM[0x1000] to MEM[0x1BFF] (3072 bytes).
- **Scale Factors:** 64 Unsigned 32-bit fixed-point scale factors are stored from MEM[0x1C00] to MEM[0x1CFF] (256 bytes).
- **CLS Token:** The 1×64 Signed INT8 vector is stored from MEM[0x2000] to MEM[0x203F] (64 bytes).
- Position Embeddings: The 65×64 Signed INT8 matrix is stored from MEM[0x2040] to MEM[0x307F] (4160 bytes).

5.3. Final Results

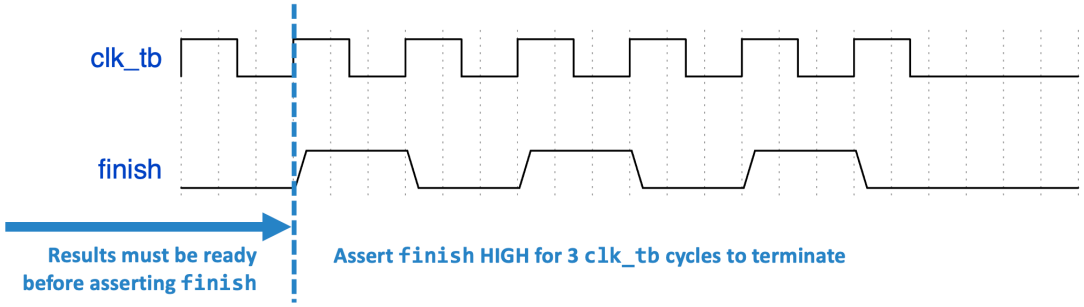
After completing all the steps, the final 65×64 Signed INT8 output matrix must be written back to the memory. The results must be stored sequentially starting from MEM[0x4000] to MEM[0x503F] (4160 bytes) in raster-scan ordering. Unassigned memory areas ("Empty" gaps) are not verified by the testbench and can be freely used as scratchpad memory.

Sample Waveform

1. Near start



2. Near termination



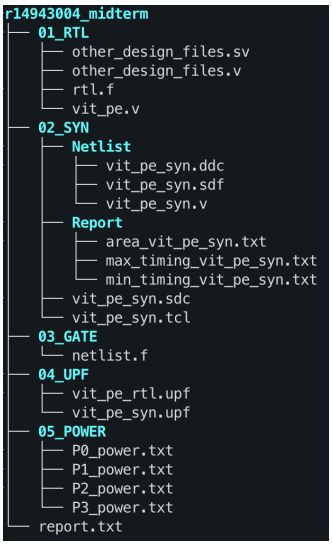
Submission

1. Create a folder named **<studentID>_midterm** and follow the hierarchy below.
2. Compress the folder **<studentID>_midterm** into a **.tar.gz** file named **acvsd-xx-midterm(-v*k*).tar.gz**. TA will only check the lastest version after the homework deadline.

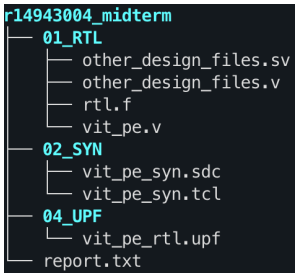
```
tar -zcvf acvsd-xx-hw1.tar.gz <path to <studentID>_midterm>    (Workstation)
```

```
tar -zcvf acvsd-xx-hw1-vk.tar.gz <path to <studentID>_midterm>    (NTU COOL)
```

3. Submit to
    a. NTU COOL
    b. The home directory (~) of the workstation of your ADFP account.



Submission - Workstation



Submission – NTU COOL

Note:

a. Use **lowercase** for all the letters. (e.g. `acvsd-00-(-vk)-midterm.tar.gz`)
b. Remember to add -v*k* (*k* is the number of version, *k* = 1, 2, …) for files submitted to NTU COOL. (e.g. `acvsd-00-midterm-v1.tar.gz`)
c. Both standard Verilog (`.v`) and SystemVerilog (e.g. `vit_pe.sv`) are fully allowed in the `01_RTL` directory
d. `04_UPF` files are optional, and no submission of UPF files will not cause any penalty.
e. Files uploaded to the ADFP workstation must be kept as a copy in your local folder or computer.

Grading Policy

1. TA will run your code with the following format of command. Make sure to run this command with no error message on the ADFP server.

    RTL simulation:

```
vcs -full64 -R -f rtl.f -sverilog -v2005 +v2k +define+PAT_NUM=i
+notimingcheck
```

Gate-level simulation:

```
vcs -full64 -R -f netlist.f +v2k -sverilog -v2005
+define+SDF+PAT_NUM=i +maxdelays -negdelay +neg_tchk
```

where *i* is the number of pattern.

Note: You can `+define+FSDB` and add `-debug_access+all` to dump `.fsdb` file.

2. Simulation: **80%**

	Public must pass all patterns to receive any score	**Private** graded proportionally based on the pass rate
RTL	15%	15%
Gate-level	25%	25%

Note:

a. Zero runtime timing violations are allowed after reset de-assertion.
b. **Total simulation time must not exceed 2.5 × 10^6 ns (2.5 ms).**

3. Performance: **20%**

$$\text{score} = \left(\sum_{i=0}^{3} \text{power}_i \times \text{time}_i \right) \times \text{area}^2$$

Note:

a. Unit: power (mW), time (ns), area (μm^2)
b. There is no hard baseline for performance scores in this assignment.
c. You are encouraged to challenge yourself to optimize hardware area.

4. 5 points will be detected if there is any incorrect file naming or directory structure.
5. 0 point for gate-level simulations and performance sections if the setup/hold slack is negative.
6. If other specification violations occur with all simulations passed, performance score will be multiplied by 0.7.
7. TA will run simulation with UPF if submitted.
8. No late submission, otherwise 0 point for this assignment.
9. No plagiarism. Plagiarism in any form, including copying from online sources, is strictly prohibited.

References

[1] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021. [Online]. Available: https://arxiv.org/abs/2010.11929